

## Introduction to NoSQL

NoSQL database means “Not Only SQL”, Non-Relational, Non tabular database and Schema less database It is designed to handle Large volumes of data (**Big Data**), High-speed read/write operations, Distributed and scalable applications

### Key Features:

- Schema-less or flexible schema
- Horizontal scalability (scale-out)
- High availability
- Designed for unstructured/semi-structured data

### Types of NoSQL Databases:

1. **Key-Value Stores** → e.g., Redis
2. **Document Stores** → e.g., MongoDB
3. **Column-Family Stores** → e.g., Cassandra
4. **Graph Databases** → e.g., Neo4j

### Characteristics of NoSQL Databases

#### 1.Schema-less Design

- No predefined structure
- Fields can vary between records

#### 2. Distributed Architecture

- Data stored across multiple servers

#### 3. High Scalability

- Easily scale out by adding nodes

#### 4. High Availability

- Replication ensures minimal downtime

#### 5. Flexible Data Model

- Supports structured, semi-structured, unstructured data

### Difference between SQL and NoSQL

| SQL                                                  | NoSQL                                                                  |
|------------------------------------------------------|------------------------------------------------------------------------|
| Tables                                               | Collections                                                            |
| Rows                                                 | Documents                                                              |
| Columns                                              | Fields                                                                 |
| Joins                                                | No joins → improves performance                                        |
| ACID (Atomicity, Consistency, Isolation, Durability) | BASE Approach (Basically Available, Soft State, Eventual consistency ) |

## CAP Theorem (Core Concept)

NoSQL systems are based on the **CAP Theorem**, which states:

A distributed system can provide only **two out of three**:

- **Consistency (C)** → Same data everywhere
- **Availability (A)** → Always responsive
- **Partition Tolerance (P)** → Works despite network failure

## Advantages of NoSQL

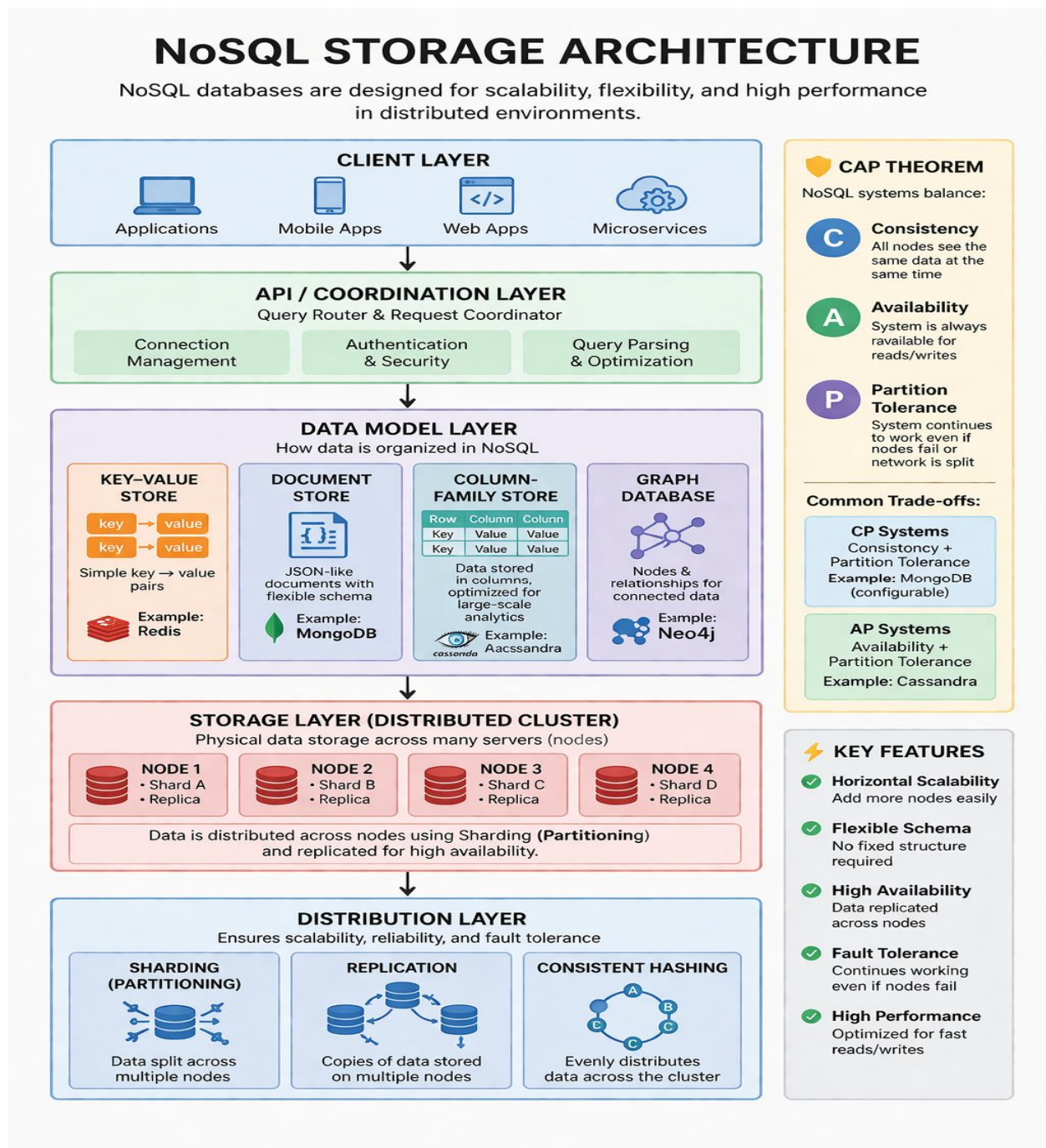
1. High scalability
2. Fast performance
3. Flexible schema
4. Handles big data efficiently
5. Suitable for cloud and distributed systems

## Use Cases of NoSQL

1. Social media platforms
2. Real-time analytics
3. IoT applications
4. E-commerce platforms
5. Content management systems

## What is NoSQL Storage Architecture.

NoSQL storage architecture refers to how **non-relational databases** store, organize, and retrieve data. Unlike traditional relational databases (tables + rows), NoSQL systems are designed for **scalability, flexibility, and high performance**, especially for large-scale or unstructured data.



## 1. Client Layer:

**Client Layer** = “Who is using the database?” Like that Web Applications (e.g., browsers), Mobile Apps , Microservices, Third-party APIs.

### Functions of Client Layer

#### Sending Requests

Clients send operations such as:

- **Create** (Insert data)
- **Read** (Fetch data)
- **Update**
- **Delete**

## 2. API / Coordination Layer in NoSQL

### 1. What is the API Layer in NoSQL?

The **API (Application Programming Interface) Layer** is the interface through which:

- Applications interact with the NoSQL database
- Data operations (CRUD) are performed

It acts as a **bridge between the application and the database system.**

#### Functions of API Layer:

- Accepts user/application requests
- Translates them into database operations
- Returns results to the application

## 3. Data Model Layer in NoSQL

The **Data Model Layer** in NoSQL defines:

- **How data is structured**
- **How data is stored**
- **How data is related and accessed**

It is the **core layer** that determines how efficiently a NoSQL database works.

## 4.Storage Layer in NoSQL

The **Storage Layer** in NoSQL is responsible for:

- **Physically storing data on disk or memory**
- Managing how data is **written, read, and retrieved efficiently**

It is the **lowest layer** in NoSQL architecture and directly interacts with hardware.

## 5. Distribution Layer in NoSQL

The **Distribution Layer** in NoSQL is responsible for:

- **Distributing data across multiple nodes (servers)**
- Managing how data is **partitioned, replicated, and accessed**

It enables NoSQL databases to work as **distributed systems**, appearing as a single database to users.

### ✓ Data Partitioning (Sharding)

- Divides large datasets into smaller parts (shards)
- Each shard stored on different nodes

Improves:

- Scalability
- Performance

### ✓ Replication

- Creates multiple copies of data across nodes

Ensures:

- Fault tolerance
- High availability

### ✓ Data Distribution Strategy

Determines **how data is spread** across nodes.

## Consistent Hashing in NoSQL

### 1. What is Consistent Hashing?

**Consistent Hashing** is a technique used in distributed systems to:

- **Distribute data across multiple nodes (servers)**
- Minimize data movement when nodes are **added or removed**

It is widely used in NoSQL databases like Cassandra and DynamoDB.

### Q3. Operations in NoSQL.

Ans: Operations in NoSQL databases primarily focus on **CRUD** (Create, Read, Update, Delete) operations to perform for high-speed, distributed data handling rather than complex. They utilize flexible schemas (e.g., JSON documents, key-value pairs) and are generally BASE-compliant (Basically Available, Soft state, Eventual consistency), enabling horizontal scalability.

### Key Operations in NoSQL

- **CRUD Operations:** These are the foundational operations.
  - **Create:** Inserting data (e.g., insertOne, insertMany in MongoDB).
  - **Read:** Retrieving data using keys, indexes, or filters, which is optimized for speed.
  - **Update:** Modifying existing data, which can be done in part or in whole.
  - **Delete:** Removing data, which must be carefully managed to avoid performance issues in distributed systems.

- **Aggregation and Querying:**

While NoSQL lacks complex SQL JOINS, it uses methods like \$match (filtering), \$group, and \$sort to process data.

- **Distributed Data Operations:**

- **Sharding or Data Partitioning:** Dividing data across multiple nodes to improve scalability.
- **Replication:** Maintaining copies of data across nodes for high availability and load balancing.

## Q4. What is Query Model in NoSQL.?

Unlike SQL, NoSQL databases use different query approaches:

### Examples

1. **Key-Value Stores** → e.g., Redis
2. **Document Stores** → e.g., MongoDB
3. **Column-Family Stores** → e.g., Cassandra
4. **Graph Databases** → e.g., Neo4j

### Features

- No fixed schema
- Often API-based queries instead of SQL
- Some support query languages (e.g., MongoDB Query Language)

## Q5. What is Modifying Data Stores & Managing Evolution

### 1. Modifying Data Stores

In NoSQL databases, modifying data stores means **updating, inserting, or deleting data** in a flexible way (without fixed schema).

#### Common Operations:

- **Insert (Create)** → Add new data
- **Update (Modify)** → Change existing data
- **Delete (Remove)** → Remove data

### 2. Evolution

**Evolution** means how the database structure **changes over time** as requirements grow.

#### Why Evolution is Needed:

- New features in applications
- Changing business needs
- Adding new data fields

## Q6. What is Indexing and Ordering Data Sets

### Indexing

- Improves query performance
- Types include:
  - Single-field indexes
  - Composite indexes
  - Full-text indexes
  - Geospatial indexes

### Ordering

- Sorting can be done using indexes
- Some NoSQL systems limit sorting capabilities for performance

## Q7.How We Managing Transactions and Data Integrity in NoSQL.?

### Transactions

- Traditional ACID transactions may be limited
- Many NoSQL systems follow **BASE**:
  - Basically Available
  - Soft state
  - Eventually consistent

### Consistency Models

- Strong consistency
- Eventual consistency
- Tunable consistency (e.g., Cassandra)

### Data Integrity

- Managed at application level in many systems
- Some support limited ACID transactions (e.g., MongoDB)

### Summary

NoSQL databases are designed for:

- Scalability
- Flexibility
- High performance